Verilog Language and Application

Week 7 Lab Document

Lab Manual **Revision 1.0**

cadence®

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Table of Contents

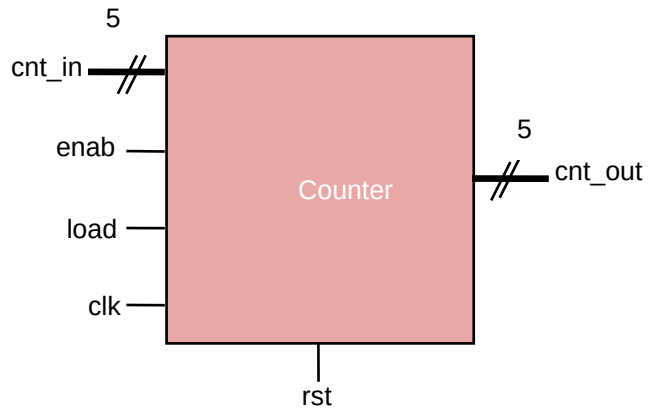
Verilog Language and Application

Lab 1 Modeling the Counter Using Functions

Objective: To encapsulate counter design combinational behaviors in a function.

You typically use a function to encapsulate an operation performed multiple times with multiple different sets of operands. Encapsulating functionality reduces code “bloat” to make it more understandable and thus more reusable.

In this lab, you create a counter design as per the specification using functions to practice the construct and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- The counter is clocked on the rising edge of `clk`.
- `rst` is active high.
- `cnt_in` and `cnt_out` are both 5-bit signals.
- If `rst` is high, output will become *zero*.
- If `load` is high, the counter is loaded from the input `cnt_in`.
- Otherwise, if `enab` is high, `cnt_out` is incremented and `cnt_out` is unchanged.

Designing a Counter

1. Change to the `lab1-func` directory and examine the following file.

`counter_test.v`	Counter test
`counter.v`	Counter module (incomplete)

2. Use your favorite editor to modify the `counter.v` file name it as "`counter`".

3. Describe the combinational logic of the counter using functional block. You can use an 'if' construct to model the counter combinational block.

Verifying the Counter Design

1. Using the test module, to test your counter design using the following command with Xcelium™.

```
xrun counter.v counter_test.v (Batch Mode)
```

or

```
xrun counter.v counter_test.v -gui -access +rwc ( GUI Mode)
```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the `-f xrun` option.

```
xrun -f filelist.txt -access rwc
```

You should see the following results.

```
At time 20 rst=0 load=1 enab=1 cnt_in=10101 cnt_out=10101
At time 30 rst=0 load=1 enab=1 cnt_in=01010 cnt_out=01010
At time 40 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 50 rst=1 load=1 enab=1 cnt_in=11111 cnt_out=00000
At time 60 rst=0 load=1 enab=1 cnt_in=11111 cnt_out=11111
At time 70 rst=0 load=0 enab=1 cnt_in=11111 cnt_out=00000
TEST PASSED
```

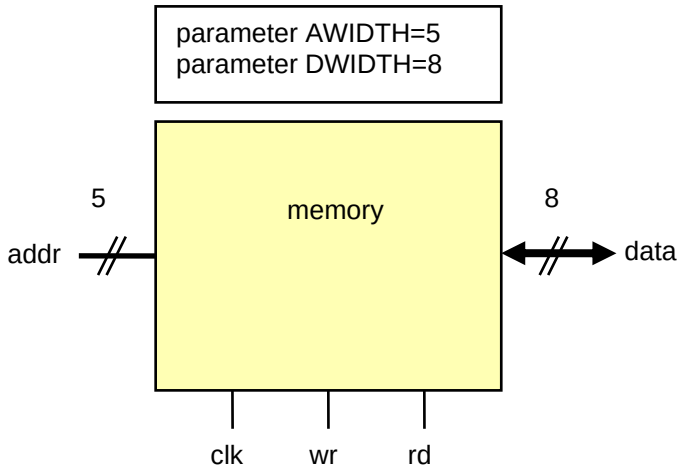
3. Correct your counter description as needed.

Lab 2 Modeling the Memory Test Block Using Tasks

Objective: To encapsulate procedural behaviors in memory test using tasks.

You typically use a function to encapsulate an operation performed multiple times with multiple different sets of operands. Encapsulating functionality reduces code “bloat” to make it more understandable and thus more reusable.

In this lab, you create a memory design as per the specification using tasks to practice the construct and verify it using the provided testbench. Please make sure to use the same variable name as the ones shown in block diagrams.



Specifications

- `addr` is parameterized to 5 and `data` is parameterized to 8.
- `wr` (write) and `rd` (read) are single-bit input signals.
- The memory is clocked on the rising edge of `clk`.
- Analyze memory read and write operation timing diagram given in *Lab 9-1*.

Designing Memory Using Tasks

1. Change to the `lab1-task` directory and examine the following files.

`memory.v`	Memory module
`memory_test.v`	Memory test (incomplete)

2. Use your favorite editor to modify the `memory_test.v` file to code the write procedure and read procedure using *tasks*.

3. In the memory test block, call the appropriate read and write tasks to verify the memory block.

   Note: The statement sets that you code read and write procedures given the following task calls.

   ```
   For write: wr=1; rd=0; memory_test.addr=addr; rdata=data; @(negedge
      clk);

   For read:  wr=0; rd=1; memory_test.addr=addr; rdata='bz; @(negedge
      clk) expect(data);
   ```

Verifying the Counter Design

1. Using the provided test module, check your counter design using the following command with Xcelium™.

   ```
   xrun memory.v memory_test.v (Batch Mode)
   ```

   or

   ```
   xrun memory.v memory_test.v -gui -access +rwc ( GUI Mode)
   ```

2. You might find it easier to list all the files and simulation options in a text file and pass the file into the simulator using the –f xrun option.

   ```
   xrun -f filelist.txt -access rwc
   ```

   You should see the following results.

   ```
   330 addr=11111, exp_data= 00000000, data=00000000
   340 addr=11110, exp_data= 00000001, data=00000001
   350 addr=11101, exp_data= 00000010, data=00000010
   360 addr=11100, exp_data= 00000011, data=00000011
   370 addr=11011, exp_data= 00000100, data=00000100
   380 addr=11010, exp_data= 00000101, data=00000101
   390 addr=11001, exp_data= 00000110, data=00000110
   400 addr=11000, exp_data= 00000111, data=00000111
   410 addr=10111, exp_data= 00001000, data=00001000
   420 addr=10110, exp_data= 00001001, data=00001001
   430 addr=10101, exp_data= 00001010, data=00001010
   440 addr=10100, exp_data= 00001011, data=00001011
   450 addr=10011, exp_data= 00001100, data=00001100
   460 addr=10010, exp_data= 00001101, data=00001101
   470 addr=10001, exp_data= 00001110, data=00001110
   ```

```
480 addr=10000, exp_data= 00001111, data=00001111
490 addr=01111, exp_data= 00010000, data=00010000
500 addr=01110, exp_data= 00010001, data=00010001
510 addr=01101, exp_data= 00010010, data=00010010
520 addr=01100, exp_data= 00010011, data=00010011
530 addr=01011, exp_data= 00010100, data=00010100
540 addr=01010, exp_data= 00010101, data=00010101
550 addr=01001, exp_data= 00010110, data=00010110
560 addr=01000, exp_data= 00010111, data=00010111
570 addr=00111, exp_data= 00011000, data=00011000
580 addr=00110, exp_data= 00011001, data=00011001
590 addr=00101, exp_data= 00011010, data=00011010
600 addr=00100, exp_data= 00011011, data=00011011
610 addr=00011, exp_data= 00011100, data=00011100
620 addr=00010, exp_data= 00011101, data=00011101
630 addr=00001, exp_data= 00011110, data=00011110
TEST PASSED
```

3. Correct your memory test as needed.

Lab 3 Verifying the VeriRISC CPU Design

Objective: To test or verify the VeriRISC CPU.

At the *behavioral* level of abstraction, you code behavior with no regard for an actual hardware implementation, so you can utilize any Verilog construct. The behavioral level of abstraction is useful for exploring design architecture and especially useful for developing test benches. As both uses are beyond the scope of this training module, it only briefly introduces testbench concepts.

Read the specification first and then follow the instructions provided.

Specifications

The CPU architecture is as follows:

- The Program Counter (`counter`) provides the program address.
- The MUX (`mux`) selects between the program address or the address field of the instruction.
- The Memory (`memory`) accepts data and provides instructions and data.
- The Instruction Register (`register`) accepts instructions from the memory.
- The Accumulator Register (`register`) accepts data from the ALU.
- The ALU (`alu`) accepts memory and accumulator data, and the opcode field of the instruction, and provides new data to the accumulator and memory.

If all components of the CPU are working properly, it will:

1. Fetch an instruction from the memory.
2. Decode the instruction.
3. Fetch a data operand from memory if required by the instruction.
4. Execute the instruction, processing mathematical operations, if required.
5. Store results back into either the memory or the accumulator. This process is repeated for every instruction in a program until an `HLT` instruction is found.

Verifying the VeriRISC CPU Design

1. Change to the `lab2-risc` directory and examine the following files.

`risc.v`	RISC model
`risc_test.v`	RISC test (incomplete)
`files.txt`	List of RISC sources

`CPUtest1.txt`
`CPUtest2.txt`
`CPUtest3.txt`
	Diagnostic Test programs

`restore.tcl`	Simulation setup files

2. Review the supplied testbench in the file `risc_test.sv`. The testbench verifies your CPU design using three diagnostic programs as follows.

   CPUtest1.txt (Basic Test) – This diagnostic program tests the basic instruction set of the VeriRisc system. If the system executes each instruction correctly, then it should halt when the HLT instruction at address 17(hex) is executed. If the system halts at any other location, then an instruction did not execute properly. Refer to the comments in this file to see which instruction failed.

   CPUtest2.txt (Advanced Test) – This diagnostic program tests the advanced instruction set of the VeriRisc system. If the system executes each instruction correctly, then it should halt when the HLT instruction at address 10(hex) is executed. If the system halts at any other location, then an instruction did not execute properly. Refer to the comments in this file to see which instruction failed.

   CPUtest3.txt (Fibonacci Calculator) – This is an actual program that calculates the Fibonacci number sequence from 0 to 144. The Fibonacci number sequence is a series of numbers in which each number in the sequence is the sum of the preceding two numbers (i.e., 0, 1, 1, 2, 3, 5, 8, 13 ...). If all the instructions execute correctly, the CPU will encounter an HLT instruction at Program Counter address 0x0C. If the CPU halts at some other address, then an instruction did not execute properly. Refer to the comments in this file to see which instruction failed.

3. Use your favorite editor to modify the `risc_test.v` file to complete the RISC verification environment to verify the *JMP* instructions only. In the file comments are mentioned where code needs to be added; also, you can refer to the SKZ instruction.

   Please follow the comments in the `risc_test.v` file.

4. Using the following command with Xcelium, simulate the design and testbench.

```
xrun -f files.txt (Batch Mode)
```

You should see the following results:

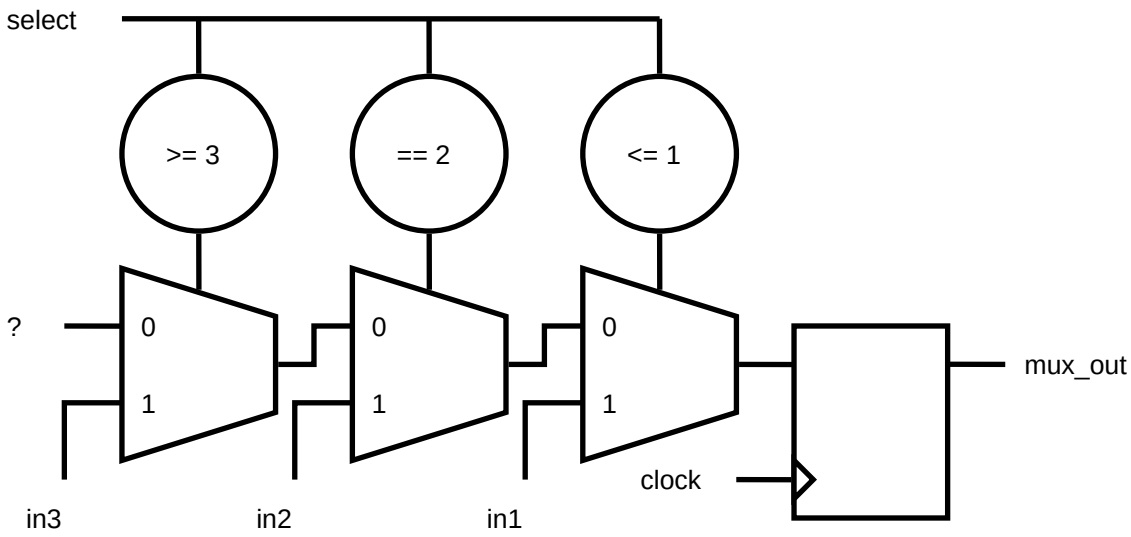
```
Testing reset
Testing HLT instruction
Testing JMP instruction
Testing SKZ instruction
Testing LDA instruction
Testing STO instruction
Testing AND instruction
Testing XOR instruction
Testing ADD instruction
Doing test CPUtest1.txt
Doing test CPUtest2.txt
Doing test CPUtest3.txt
TEST PASSED
```

5. Correct your RISC test modifications as needed.

Lab 4 Exploring the Synthesis Process

Objective: To briefly observe the synthesis process and examine the results.

In this lab, you synthesize a small multiplexor model and examine the synthesis results



Important Information

- The multiplexor model conditionally uses either a *case* statement or an *if* statement.
- If you specify neither of them, then it uses an *if* statement. If you inadvertently specify both, it uses a *case* statement.

Synthesizing a Generic Counter Design

1. Change to the `lab3-muxsynth` directory and examine the following files.

`mux.v`	Multiplexor model
`mux_test.v`	Multiplexor test
`genus_shell.tcl`	Synthesis script

2. Verify the RTL model using the the commands with Xcelium™.

   ```
   xrun mux.v mux_test.v (Batch Mode)
   ```

   You should see the below results.

   ```
   time=15 select=00 in1=0 in2=x in3=x mux_out=0
   time=25 select=00 in1=1 in2=x in3=x mux_out=1
   time=35 select=01 in1=1 in2=x in3=x mux_out=1
   time=45 select=01 in1=0 in2=x in3=x mux_out=0
   time=55 select=10 in1=x in2=0 in3=x mux_out=0
   time=65 select=10 in1=x in2=1 in3=x mux_out=1
   time=75 select=11 in1=x in2=x in3=1 mux_out=1
   time=85 select=11 in1=x in2=x in3=0 mux_out=0
   TEST PASSED
   ```

3. Synthesize the RTL model by using the Genus™ Synthesis Solution tool.

   ```
   genus -f genus_shell.tcl
   ```

   Check to see that the *synthesis succeeded*.

   The script writes a pre-synthesis netlist and a post-synthesis netlist.

4. In your favorite editor, examine the *mux.vs* pre-synthesis netlist and attempt to correlate its contents with the multiplexor behavioral description.

   How many multiplexors does it contain?

   Answer: _____

   How many operators does it contain?

   Answer: _____

   How many latches does it contain?

   Answer: _____

5. In your favorite editor, examine the *mux.vg* post-synthesis netlist and attempt to correlate its contents with the multiplexor behavioral description.

   How many multiplexors does it contain?

   Answer: _____

   How many operators does it contain?

   Answer: _____

   How many latches does it contain?

   Answer: _____

6. Verify the gate-level model using the following commands with Xcelium.

   ```
   xrun mux.vg mux_test.v -v ../tutorial.v -vlogext vg
   ```

   You should see `TEST PASSED` with similar results as above.

7. Save the `mux.vs and mux.vg` as `mux_ifdef.vs` and `mux_ifdef.vg` for future comparison with the `USE_CASE` version.

8. Again, verify the RTL model but this time use the following commands with Xcelium.

   ```
   xrun mux.v mux_test.v -define USE_CASE
   ```

   You should see `TEST PASSED` with similar results as above.

9. Synthesize the design again and examine the pre-synthesis and post-synthesis netlists.

   *For this model, does coding style significantly affect the **pre**-synthesis netlist?*

   Answer: _____

   *For this model, does coding style significantly affect the **post**-synthesis netlist?*

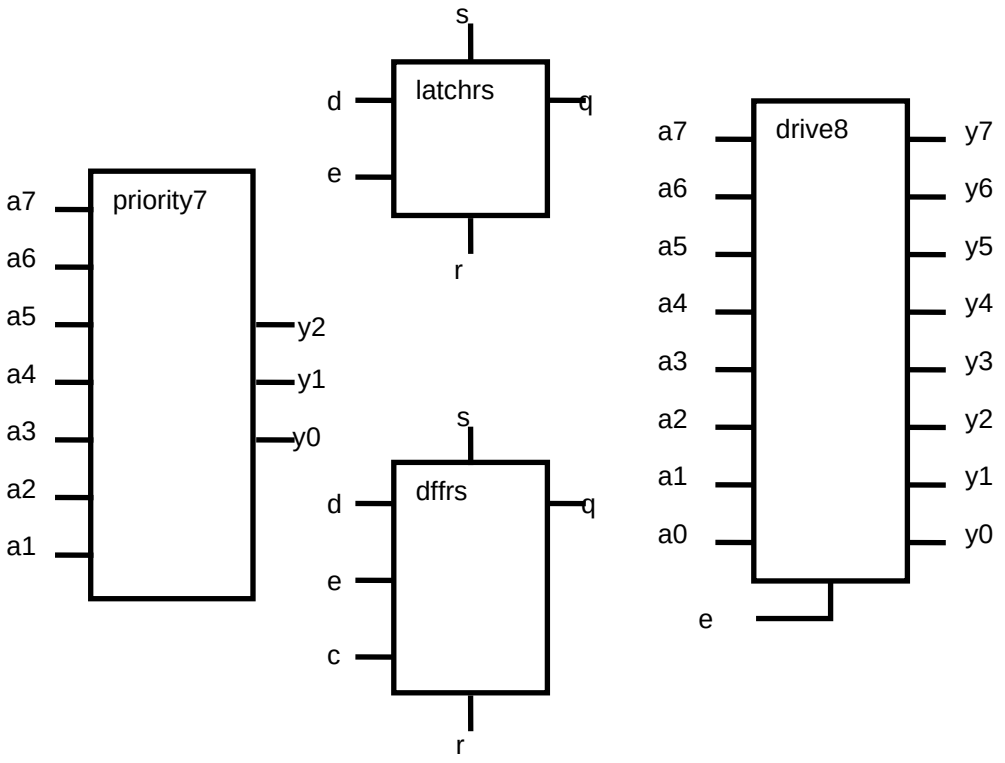
   Answer: _____

10. Optionally verify the gate-level model again.

Lab 5 Using a Component Library

Objective: To demonstrate mastery of basic coding styles and to code and synthesize some representative models.

In this lab, you will code and synthesize some representative models.



Synthesizing a Generic Modules

1. Change to the `lab4-lib` directory and examine the following files.

`lib.v`	Model library (incomplete)
`lib_test.v`	Model library tests
`genus_shell.tcl`	Synthesis script

Note: To facilitate your development effort, the library and tests can conditionally compile and test individual models. You specify the models and tests to compile by defining some combination of the `priority7`, `latchrs`, `dffrs`, and `drive8` macros on the invocation command line, as described in the previous lab. If you do not specify any, then all are compiled and tested.

2. In your favorite editor, modify the `lib.v` file to complete the model definitions. Set and reset are asynchronous and active low. You can elect to individually complete and test each model.

   Note: From the latch template, the synthesis tool cannot infer an asynchronous set or reset. The latch model includes the `async_set_reset` pragma to specify the signals to directly connect to the component's asynchronous set and reset pins.

3. Verify the RTL model(s) using the following commands with Xcelium™.

   a. Fill in the macro name (or omit the option to test all models).

   ```
   xrun lib.v lib_test.v -define macrolib (Batch Mode)
   ```

   You should see the below:

   ```
   TEST PASSED - DRIVER
   TEST PASSED - PRIORITY
   TEST PASSED - LATCH
   TEST PASSED - DFF
   ```

4. Correct your models until all pass their tests.

5. When all models pass their test, synthesize the RTL models by using the Genus™ Synthesis Solution tool.

   ```
   genus -f genus_shell.tcl
   ```

   a. The script writes a separate post-synthesis netlist for each model.

   b. Check to see that the *synthesis succeeded*.

   c. Correct your models until all can synthesize.

6. Verify the gate-level models using the following commands with Xcelium.

   ```
   xrun *.vg lib_test.v -v ../tutorial.v -vlogext vg -define macrolib
   ```

   a. Fill in the macro name (or omit the option to test all models).

b. You should see the following:

```
TEST PASSED - DRIVER
TEST PASSED - PRIORITY
TEST PASSED - LATCH
TEST PASSED - DFF
```

c. If any model fails its test, then you can ask the instructor for help or try again after you study the lecture module *Avoiding Simulation Mismatches*.